

Roots That Are Always There

The virtual roots of a real polynomial, machine-checked in Lean 4

Carles Marín*

Independent researcher
karlesmarin@gmail.com

June 2026

Abstract

A real polynomial of degree d may have fewer than d real roots, but it always has exactly d *virtual roots*: continuous, semialgebraic substitutes $\rho_{d,1}(p) \leq \dots \leq \rho_{d,d}(p)$ that coincide with the real roots when these exist and otherwise sit where the polynomial comes closest to vanishing. They were introduced by González-Vega, Lombardi and Mahé [1] and are the natural carrier of the Budan–Fourier count [2]. I report a complete, **sorry**-free formalization in Lean 4 over Mathlib of their *structural theory*: the recursive construction by a choice operator $\mathcal{R}_d$ that minimises $|p|$ on each interval where p' keeps a constant sign; the fundamental count, that there are exactly d of them (`vroots_length`); that they come out sorted and inside the bracketing interval (`vroots_isChain`, `vroots_subset_Icc`); that $\mathcal{R}_d$ returns an actual root wherever p has one (`R_eval_eq_zero_of_le_zero_le`); and the *Rolle interlacing*

$$\rho_{d,r}(p) \leq \rho_{d-1,r}(p') \leq \rho_{d,r+1}(p),$$

the statement that each virtual root of p' is wedged between two consecutive virtual roots of p (`vroots_interlacing`). Building on that structure I then prove the *exact* Budan–Fourier count itself: the number of virtual roots in $(a, b]$ equals the sign-variation drop $V(a) - V(b)$ of the derivative tower (`card_vroots_Ioc_eq_fourierVar`, Section 6)—the equality that the classical Budan–Fourier theorem only bounds for the wobbling count of *real* roots. This fuses the construction here with a second, independently recursive one, the `fourierVar` sign count of the companion Budan–Fourier formalization [18]. To the best of my knowledge—based on searches in June 2026 of Mathlib, the Coq `mathcomp-real-closed` library, the Isabelle AFP, and HOL Light (Section 9)—virtual roots, and this exact count through them, had not been formalized in any interactive theorem prover; this is their first development. Every theorem depends only on the three standard axioms `propext`, `Classical.choice`, `Quot.sound`.

What you will find here. A short guided tour, built as a string of questions, one per section (Figure 1): *how many* roots a polynomial really has, once the count is made not to wobble (§2); *how* to build the d of them through a choice operator $\mathcal{R}_d$ (§3); *why* that construction resists being made formal (§4); *how* they interlace, in the manner of Rolle (§5); and *how many* land in an interval—the exact Budan–Fourier count (§6). The development is two Lean files, `VirtualRoots.lean` (the structure) and `VirtualRootsCount.lean` (the count); the load-bearing lemmas are gathered in Section 8, the honest limits in Section 9. If you read only two things, read the interlacing (Theorem 1) and the count (Theorem 2).

*Formalized with the assistance of an AI system (Claude, Anthropic); the mathematics is classical, and every claim, scope statement, and limitation is the author’s responsibility. The boundary of the contribution—in particular what is *not* yet formalized—is set out plainly in Section 9, and the Lean kernel, not the assistant, is what certifies the proofs.

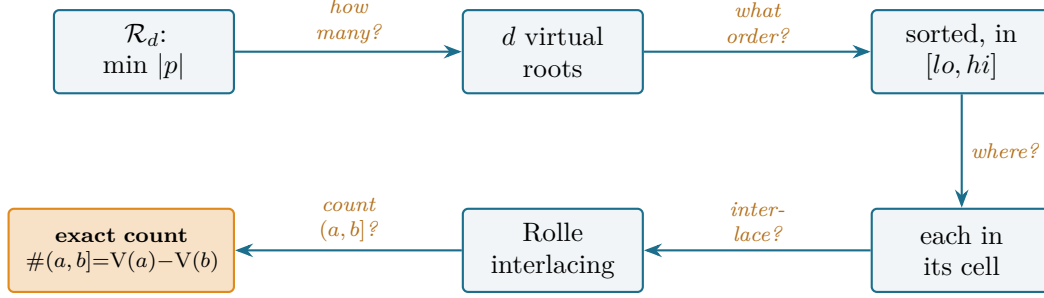


Figure 1: The reader’s map. From the choice operator $\mathcal{R}_d$ to a certified interval count; each arrow is the question its section answers, and every node is a block of **sorry-free** Lean. The clean chain—operator, d roots, order, per-cell localization, interlacing, count—is the architecture the whole development hangs on.

1 A count that does not wobble

How many roots does a polynomial have? The honest answer wobbles. A real polynomial of degree d has *somewhere between 0 and d* real roots, and the number lurches as the coefficients move: $x^2 - t$ has two real roots, then one, then none as t falls through zero, while its degree never changes. A count that jumps when nothing essential has is a poor count. **There is a better one: a degree- d polynomial always has exactly d virtual roots**—real numbers $\rho_{d,1}(p) \leq \dots \leq \rho_{d,d}(p)$ that coincide with the genuine roots when those exist and otherwise sit where the polynomial comes closest to vanishing. They never jump, never disappear, and vary continuously with the coefficients. This paper machine-checks the structural theory of that better count.

The picture to keep is the simplest non-trivial one. The polynomial $p = x^2 + 1$ has no real root, yet its graph dips to a single closest approach at $x = 0$. Virtual roots see that approach: p has the double virtual root $\rho_{2,1} = \rho_{2,2} = 0$ (Figure 2). Where a real root would be, there is a virtual one; where two conjugate roots hide, the two virtual roots collide at the foot of the dip rather than vanish. Nothing is invented—the construction is explicit and semialgebraic—and nothing is lost: **the count is always d** . The driving thread of the paper is this contrast, that real roots are a count that wobbles and virtual roots are the stable d -element family underneath, and that the stability is not asserted but *constructed*, concretely enough for a kernel to check.

The question is old, and so are its answers. Descartes (1637) bounded the positive roots by the sign changes of the coefficients; Budan and Fourier (1807–1831) read the same count off the derivative tower $p, p', p'', \dots$; Sturm (1835) made it exact, but only for *distinct* roots. Each of these rules counts a quantity that wobbles—the real roots—so Budan’s and Fourier’s only *bound* it, off by an even surplus no one could name. The fix came late: González-Vega, Lombardi and Mahé isolated the virtual roots in 1998 [1], the stable d -element family for which that bound becomes an equality (Coste, Lajous, Lombardi and Roy [2]). This paper machine-checks that family—its construction, its count, its order, and the Rolle interlacing—and then the equality itself (Section 6): the surplus, it turns out, was only ever the count looking at the wrong roots. I came to it as I came to Sturm [19] and Budan–Fourier [18]: by asking which classical results in real-root counting a mature library still lacks, and finding the virtual roots absent from every prover I could search.

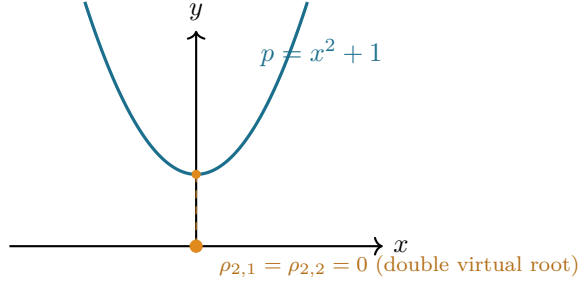


Figure 2: Roots that are always there. The polynomial $x^2 + 1$ has *no* real root, but a double virtual root at $x = 0$, where the graph comes closest to the axis. A degree-2 polynomial has two virtual roots whatever its discriminant; here they coincide. In Lean, on a bracketing interval, the construction returns the list `[0, 0]`.

2 The d roots that are always there

The idea that a polynomial carries d canonical real “roots” regardless of how many are genuine is González-Vega–Lombardi–Mahé’s [1], set in the sign-variation theory whose textbook account is Basu–Pollack–Roy [11], Ch. 2. The definition is `vroots` and the count is `vroots_length`.

Between two consecutive critical points of p —two consecutive roots of p' —the derivative keeps a constant sign, so p is strictly monotone there and $|p|$ has a single minimum. The point where that minimum is attained is a virtual root: the unique root of p in the cell if p changes sign across it, and otherwise the cell endpoint where p is smallest. The critical points of p are the virtual roots of p' , so the construction recurses on the derivative, exactly as Rolle’s theorem [6] interleaves the roots of p and p' .

Why exactly d . A degree- d polynomial has a degree- $(d-1)$ derivative, hence—by induction— $d-1$ virtual roots of p' ; these cut the line into d cells, and each cell contributes one virtual root of p . The count never degenerates: it is the degree, not the number of real roots. This is the first theorem, and the cleanest:

$$\# \{\text{virtual roots of } p\} = \deg p.$$

In Lean this is `vroots_length`: the list `vroots lo hi p` of virtual roots has length `p.natDegree`. Real roots are at most d (and can be far fewer, or none); **virtual roots are always exactly d** . The gap between the two counts is precisely the even Budan–Fourier surplus, now realized as collisions of virtual roots away from the axis.

The recipe is clean on paper—*minimise $|p|$ on each p' -monotone cell*—but “the point where $|p|$ is least” is a choice, and “the cells” are cut by an object defined by the same recursion. Both have to be made explicit before a kernel will accept them.

3 Building them: the choice operator $\mathcal{R}_d$

The recursive construction—cut the line by the critical points, minimise $|p|$ on each piece—is the definition of González-Vega–Lombardi–Mahé [1], tied to the sign-variation count by Coste [2]. Where a paper proof says “take the point where $|p|$ is least,” a formal one must produce it: this is the operator `R` (existence `exists_isMinOn_abs_eval`), assembled by the recursion `vroots`.

The cells of p are cut by the virtual roots of p' together with two outer fences. Rather than work on all of $\mathbb{R}$ from the start, I fix a closed interval $[lo, hi]$ and build the virtual roots inside it;

taking lo, hi beyond every root of the derivative tower (a Cauchy bound) recovers the global object, and that last step is discussed in Section 9. Working on $[lo, hi]$ keeps every quantity a genuine real number and lets the recursion reuse one choice operator verbatim.

The choice operator $\mathcal{R}_d$. On a closed interval $[a, b]$, the continuous function $|p|$ attains a minimum (the interval is compact); call a minimiser $\mathcal{R}_d(a, b, p)$. In Lean this is `R`, defined by choosing a minimiser whose existence is `exists_isMinOn_abs_eval`. Two facts about it carry the whole theory. First, it **lands in the interval**: `R_mem` says $\mathcal{R}_d(a, b, p) \in [a, b]$. Second, it **captures actual roots**: if $p(a) \leq 0 \leq p(b)$ (or the reverse), the intermediate value theorem gives a root, the minimum of $|p|$ is then 0, and the minimiser is that root—`R_eval_eq_zero_of_le_zero_le`. So on a cell where p changes sign, $\mathcal{R}_d$ is the genuine root; where p does not, it is the nearer endpoint. That is the precise sense in which virtual roots extend the real ones.

The choice operator $\mathcal{R}_d(a, b, p)$ (recipe)

On a closed interval $[a, b]$, take **any minimiser of $|p|$** (it exists, $[a, b]$ is compact). Two facts are all the theory needs. **It lands in the interval**, $\mathcal{R}_d(a, b, p) \in [a, b]$ (`R_mem`); this single fact yields the interlacing. **It is an actual root when p has one**: if p changes sign across $[a, b]$, the minimum of $|p|$ is 0, so $\mathcal{R}_d$ is a genuine root (`R_eval_eq_zero_of_le_zero_le`); this is how virtual roots extend the real ones. Uniqueness, where p' keeps a sign, comes free from the monotonicity bricks of Q3—so the *chosen* minimiser is in fact the only one.

The recursion. The virtual roots are then assembled by listing the breakpoints

$$\text{bps} = lo :: (\text{virtual roots of } p') ++ [hi],$$

and applying $\mathcal{R}_d$ to each consecutive pair: `vroots lo hi p = zipWith ( $\mathcal{R}_d p$ ) bps bps.tail`. A degree-0 polynomial has no virtual roots; otherwise the recursion descends to p' , terminating because $\deg p' < \deg p$ (`natDegree_derivative_lt`). The well-founded definition and the length theorem are the content of `vroots` and `vroots_length` (Figure 3).

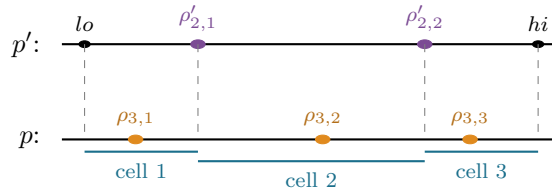


Figure 3: The recursion, for $\deg p = 3$. The two virtual roots of p' (plum) together with the fences lo, hi cut $[lo, hi]$ into three cells; on each cell p' keeps a constant sign, so $\mathcal{R}_d$ returns one virtual root of p (amber). Each $\rho_{3,j}$ lies in its own cell—this is what makes them come out sorted and interlaced with the $\rho'_{2,j}$.

4 Making the construction formal

The construction is classical and the interlacing has a one-line paper proof—“ $\mathcal{R}_d$ lands in its cell, and cells share endpoints”—so what costs the work is not new mathematics but the bookkeeping that makes a recursive, choice-defined family pass a kernel, the analogue of the list-surgery the companion Budan–Fourier development [18] needed. It enters in `vroots_chain_mem` (the master invariant) and `isChain_zipWith_R`.

A *noncomputable choice, used honestly*. $\mathcal{R}_d$ is a *chosen* minimiser, so $\mathbf{R}$ is **noncomputable** and the development leans on **Classical.choice**. That is sound and standard, but it means the virtual roots are specified, not computed: the theorems are about *a* minimiser, and what makes them unambiguous is monotonicity. The two *monotonicity bricks* shared with the companion Budan–Fourier work—`eval_strictMonoOn_of_derivative_pos` and its negative twin—say that where p' keeps a constant nonzero sign, p is strictly monotone, so $|p|$ has a *single* well and the minimiser is unique. The formal development needs only existence and the two facts `R_mem / R_eval_eq_zero`; uniqueness is what licenses the informal picture.

The breakpoints are a chain, and that is the crux. Everything downstream rests on one invariant: the augmented breakpoint list `lo :: (vroots lo hi p') ++ [hi]` is *sorted*, and every virtual root lies in $[lo, hi]$. The two halves are mutually dependent—a list is sorted only if its members are in range, and $\mathcal{R}_d$ lands in range only between sorted breakpoints—so they are proved together, by a single induction down the derivative tower (`vroots_chain_mem`). The sortedness is the engine: because $\mathcal{R}_d(\text{bps}[r], \text{bps}[r+1])$ lies in $[\text{bps}[r], \text{bps}[r+1]]$ and consecutive breakpoints are ordered, consecutive virtual roots are ordered too (`isChain_zipWith_R`). This is Rolle’s theorem in list form, and it is the part the machine would not let me wave away.

Indexing, and the small traps. Turning “each virtual root sits between its breakpoints” into a statement about `getElem` required care that a hand proof hides. Rewriting a proof-carrying indexed access (`l[r]` drags a proof $r < l.length$) is not a plain `rw`; it goes through `simp` with the index lemmas, relying on proof irrelevance to reconcile the bounds. And a destructuring `rcases` on an equation $w = lo$ silently *substitutes* `lo` away in that branch—a one-character fix (`le_refl` for `le_refl lo`), but the kind of thing only a kernel catches. These are not deep, but they are exactly the hand-waves a proof assistant refuses.

The chain and the per-index localization, once proved, are exactly what the interlacing rests on.

5 The Rolle interlacing

That the roots of p and p' interleave is Rolle’s, from 1690 [6]; that the *virtual* roots interleave the same way—even when no real roots exist—is the González-Vega–Lombardi–Mahé refinement [1] and the structural core of Coste’s count theorem [2]. It is `vroots_interlacing`, resting on the per-index localization `vroots_getElem_mem`.

The interlacing of the virtual roots of p with those of p' :

Theorem 1 (Rolle interlacing of virtual roots; `vroots_interlacing`). *For $lo \leq hi$ and each index r in range,*

$$\rho_{d,r}(p) \leq \rho_{d-1,r}(p') \leq \rho_{d,r+1}(p),$$

i.e. $(\text{vroots } lo \ hi \ p)[r] \leq (\text{vroots } lo \ hi \ (\text{derivative } p))[r] \leq (\text{vroots } lo \ hi \ p)[r+1]$.

The proof is two applications of the localization `vroots_getElem_mem`—each virtual root of p lies between two breakpoints—together with the identity that the interior breakpoints *are* the virtual roots of p' . It is the discrete shadow of Rolle’s theorem, and it places the virtual-root family on the same footing as the interlacing families that run through the real-rootedness literature—Newton’s inequalities, the Heilmann–Lieb matching polynomial—where a sequence and its derivative interleave. That this now holds for a family defined even when no real roots exist is the point.

What it is the road to. The interlacing is the structural half of the Budan–Fourier story. The arithmetic half—that the number of virtual roots in a half-open interval $(a, b]$ equals the

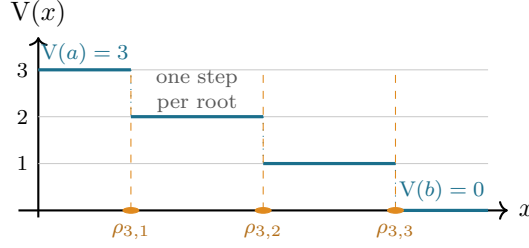


Figure 4: The exact count as a staircase. For $p = x^3 - x$ the Budan–Fourier count $V(x)$ is flat between virtual roots and drops by exactly one at each; the number of virtual roots in $(a, b]$ is the total descent $V(a) - V(b)$. Here $V(a) - V(b) = 3$ for any $a < -1$ and $b > 1$, matching the three virtual roots $\rho_{3,1} < \rho_{3,2} < \rho_{3,3}$ (all real for this p). A degree-two collision, as in Figure 2, would appear as a single step of height two—the count still exact, now reading a virtual root the real-root count cannot see.

sign-variation drop $V(a) - V(b)$ of the derivative tower [2]—is the equality that the classical Budan–Fourier inequality [18] only bounds. It fuses this construction with a second, independently recursive one, and so was the sequel; it is the subject of the next section, now proved.

6 The exact count

The interlacing was, in the first version of this work, where the paper stopped: the structural theory was proved and the arithmetic count was named honestly as the sequel. It is now proved, and this section reports it. It is the equality the whole construction was for.

Theorem 2 (The exact Budan–Fourier count; `card_vroots_Ioc_eq_fourierVar`). *Suppose every root of every member of the derivative tower $p, p', p'', \dots$ lies strictly inside $[lo, hi]$. Then for $a \leq b$ in $[lo, hi]$,*

$$\#\{\rho \text{ virtual root of } p : a < \rho \leq b\} = V(a) - V(b),$$

where $V = \text{fourierVar}$ is the Budan–Fourier sign-variation count of the tower at a point.

In Lean the statement reads, verbatim (implicit type binders $\{lo\ hi : \mathbb{R}\}$, $\{p : \mathbb{R}[X]\}$ elided):

```
theorem card_vroots_Ioc_eq_fourierVar (hp : p ≠ 0)
  (hbr : Brackets lo hi p) (ha : a ∈ Ico lo hi) (hb : b ∈ Ico lo hi) (hab :
a ≤ b) :
  ((vroots lo hi p : Multiset ℝ).filter (fun r => a < r ∧ r ≤ b)).card
  = fourierVar p a - fourierVar p b
```

The classical Budan–Fourier theorem [18] bounds the number of *real* roots in $(a, b]$ by $V(a) - V(b)$, off by an even surplus. Replace the real roots by the virtual ones and the inequality becomes an equality—the surplus was never a defect of the count, only of *which* roots one was counting. The wobbling count of Section 1 is what made Budan–Fourier an inequality; the stable count makes it exact (Figure 4).

Why it is one theorem and not a line. It fuses the two recursions the development is built from. Both V and `vroots` descend the same tower $p \rightarrow p' - V$ by peeling the leading sign, `vroots` by the Rolle interlacing—but they are *a priori* different objects, and the theorem is that they march in step. The proof goes through the reformulation

$$V_p(x) = \#\{\text{virtual roots of } p \text{ strictly above } x\}$$

(`fourierVar_eq_card_vroots_gt`, by induction down the tower), from which the $(a, b]$ count follows by subtraction; the interval theorem above is then a multiset partition. The inductive step is a single per-step lemma (`count_step`): from p' to p , the count of virtual roots above x rises by exactly the Fourier increment $V_p(x) - V_{p'}(x)$, and that increment is 0 or 1.

Where the two bits meet. Locate x in its cell $[\text{bps}[i], \text{bps}[i + 1]]$ of the p' -breakpoint partition (Figure 3). The interlacing already forces every cell strictly above x to contribute a virtual root of p and every cell below to contribute none, so the entire count increment is whether the one virtual root v_i in x 's own cell lies above x (`count_diff_eq_cell`). On that cell p' is root-free—a root of p' would be a virtual root of p' , i.e. a breakpoint, and none lies strictly between two consecutive breakpoints (Coste's Prop. 2.2, here `root_mem_vroots`)—so p is strictly monotone across it, and its $|p|$ -minimiser v_i sits above x exactly when $p(x)$ has the sign that has not yet “turned the corner” toward the root (`lt_R_iff_eval_neg` and its twin).

The one analytic step. On the Fourier side the increment is 1 exactly when $\text{sign } p(x)$ disagrees with the first surviving sign d of the tower $p', p'', \dots$ at x . The bridge between the two sides is one identity:

$$d = \text{the sign of } p' \text{ on } x\text{'s cell.}$$

The lowest non-vanishing derivative at x fixes p' just to the right of x , and on the root-free cell that sign is constant; so d is the cell's monotonicity direction. (In Lean this is `fourier_head_right`, reusing `signs_right_eq_Rseq` of the companion work [18]: the sign pattern just right of x is the leading-surviving pattern at x .) This identity is what removes the multiplicity bookkeeping that makes the classical proof delicate: once d is pinned to the cell, “ v_i above x ” and “a new Fourier variation” are the *same* event, and the two recursions agree term by term. Summing the steps gives Theorem 2 (Figure 4).

What it needs, and what it does not. The bracketing hypothesis—all tower roots strictly inside $[lo, hi]$ —is exactly what makes the count clean: $V(lo)$ is the degree, $V(hi) = 0$, and every virtual root is interior. It is the same fixed-interval discipline as the construction (Section 3); lifting it to all of $\mathbb{R}$ by a Cauchy bound is the same deferred step (Section 9). No root is computed and no isolator is built—the theorem certifies the count, not a procedure.

7 What a certified virtual-root theory enables

A machine-checked virtual-root family is the missing middle term between two things Mathlib already has and one it did not. It has Descartes' rule (the coefficient sign count) and, in the companion work, the Budan–Fourier *inequality*; it did not have the object that makes that inequality an equality. Virtual roots are that object, and Section 6 closes the equality. Downstream, they are the right input to a certified real-root *isolation* procedure in the line of Vincent, Collins and Akritas [10, 9, 8] and the sign-variation theory of Basu–Pollack–Roy [11]—the count is now exact, so a subdivision method reading $V(a) - V(b)$ on each half has, in principle, a verified guarantee to build on. And because the family is defined for every polynomial, real-rooted or not, it connects the interval methods to the algebra of the discriminant: a pair of virtual roots collides exactly when two real roots are about to become complex, which is where the even Budan–Fourier surplus comes from (Figure 5). The structural theory formalized here is the foundation all of that stands on.

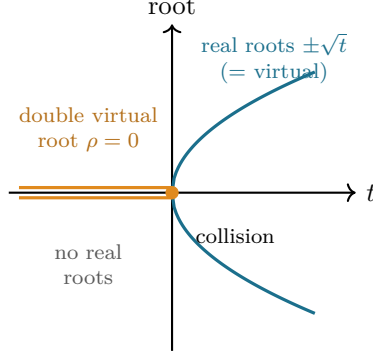


Figure 5: Roots that are always there, as a parameter moves. For $p_t = x^2 - t$ the two real roots $\pm\sqrt{t}$ (teal) exist only for $t \geq 0$ and vanish as t drops below 0; the two *virtual* roots (amber) persist for *all* t , coinciding with the real roots when $t \geq 0$ and colliding into the double virtual root $\rho = 0$ for $t < 0$. The real-root count jumps $2 \rightarrow 0$ at $t = 0$; the virtual-root count stays 2. The collision at $t = 0$ is exactly where a conjugate pair forms—the source of the even Budan–Fourier surplus. (A different view of the same phenomenon as Figure 2, now in motion.)

8 The building blocks

Table 1 gathers the load-bearing results, all `sorry-free` in `VirtualRoots.lean`; Table 2 measures the development. The dependency is short: the monotonicity bricks give the existence and the two properties of $\mathcal{R}_d$; the combined induction `vroots_chain_mem` proves sortedness and range together; and the per-index localization gives the interlacing.

Lean name	Statement	Status
<code>R_mem</code>	$\mathcal{R}_d(a, b, p) \in [a, b]$	proved
<code>R_eval_eq_zero_of_le_zero_le</code>	$p(a) \leq 0 \leq p(b) \Rightarrow p(\mathcal{R}_d(a, b, p)) = 0$	proved
<code>vroots_length</code>	$\#\{\text{virtual roots of } p\} = \deg p$	proved
<code>vroots_isChain</code>	$lo :: (\text{vroots } p) ++ [hi]$ is sorted	proved
<code>vroots_subset_Icc</code>	every virtual root lies in $[lo, hi]$	proved
<code>vroots_getElem_mem</code>	$\text{bps}[r] \leq \rho_{d,r}(p) \leq \text{bps}[r+1]$	proved
<code>vroots_interlacing</code>	$\rho_{d,r}(p) \leq \rho_{d-1,r}(p') \leq \rho_{d,r+1}(p)$	proved
<code>fourierVar_eq_card_vroots_gt</code>	$V_p(x) = \#\{\text{virtual roots } > x\}$	proved
<code>count_step</code>	per-step bridge $V_p - V_{p'} = \text{cell increment}$	proved
<code>card_vroots_loc_eq_fourierVar</code>	$\# \text{ in } (a, b] = V(a) - V(b)$	proved

Table 1: The theory of virtual roots, all `sorry-free`: the structural results in `VirtualRoots.lean` (top) and the exact count in `VirtualRootsCount.lean` (bottom), Lean 4 / Mathlib. `#print` axioms on `vroots_interlacing` and on `card_vroots_loc_eq_fourierVar` reports only `propext`, `Classical.choice`, `Quot.sound`.

Quantity	Value
Lines: <code>VirtualRoots.lean</code> / <code>VirtualRootsCount.lean</code>	378 / 944
Theorems + defs: structural / count	16+2 / 33
Mathlib modules used (both files, distinct)	12
<code>sorry</code> / <code>admit</code> / <code>native_decide</code>	0
Axioms beyond Lean’s core	0 (only <code>propext</code> , <code>Classical.choice</code> , <code>Quot.sound</code>)
Compile, warm Mathlib (<code>VirtualRoots</code> / count)	$\approx$ 8s / 9s
Toolchain	Lean 4 (godsil pin v4.30) over Mathlib

Table 2: The development measured. Build: `lake env lean VirtualRoots.lean` and `VirtualRootsCount.lean`, exit 0; compile time is for checking each file against a prebuilt Mathlib. The footprint is deliberately light—no heavy algebra, because the construction is elementary. The twelve Mathlib modules are `Analysis.Calculus.Deriv.Polynomial`, `Deriv.MeanValue`, `Topology.Algebra.Polynomial`, `Topology.Order.Compact`, `Topology.Order.IntermediateValue`, `Algebra.Polynomial.Derivative`, `Algebra.Polynomial.Roots`, `Algebra.Polynomial.FieldDivision`, `Data.List.Destutter`, `Data.Sign.Basic`, `Data.Real.Basic` and `Algebra.Ring.Parity`.

9 The boundary of the contribution

What is proved. The structural theory of the $\mathcal{R}_d$ virtual roots, sorry-free: existence and the count ($= \deg p$), that they are sorted and lie in the bracketing interval, that $\mathcal{R}_d$ returns an actual root wherever p changes sign, and the Rolle interlacing (`VirtualRoots.lean`); and, built on these, the exact Budan–Fourier count $\#\{(a, b]\} = V(a) - V(b)$ (`VirtualRootsCount.lean`, Section 6). These are the results of González-Vega–Lombardi–Mahé [1] and Coste [2] in their structural and counting parts. The count is proved on a bracketing interval $[lo, hi]$ that strictly contains every tower root.

What is not proved. Two things, both genuinely beyond the present files. *Continuity*: that the virtual roots vary continuously with the coefficients—half of why González-Vega–Lombardi–Mahé introduced them [1]—is not touched (Section 10). *The global object*: the construction and the count are on a fixed bracketing interval $[lo, hi]$; recovering the object on all of $\mathbb{R}$ by taking lo, hi beyond every root of the derivative tower (a Cauchy bound, available in Mathlib as `Polynomial.cauchyBound`) is routine but not carried out here.

Novelty. To the best of my knowledge, neither virtual roots nor the exact Budan–Fourier count through them had been formalized in any interactive theorem prover before this. The claim rests on searches in June 2026 of: Mathlib (no virtual roots; it has Descartes via `Polynomial.signVariations`); the Coq `mathcomp-real-closed` library (actual roots and the Thom encoding, not virtual roots); the Isabelle AFP, whose `Budan_Fourier` entry formalizes the inequality with multiplicity but not the virtual-root object nor the equality; and HOL Light (Sturm and Descartes, not virtual roots). I make this claim at the level of the named object; I cannot exclude an unpublished or differently-named development, and I would welcome a pointer.

Authorship. The mathematics is classical, due to the authors cited. The formalization was carried out in close interaction with an AI assistant (Claude, Anthropic): the choice of target, the overall proof architecture, the statements, and the scope and novelty claims are mine; the assistant drafted and iterated much of the Lean under that direction. The division does not bear on validity—a proof is accepted only when the Lean kernel type-checks it, which it does, with the axiom profile reported above—but it is stated plainly here for scientific attribution.

10 What remains open

Three facts mark where the work points next; the first is now settled and names its own remaining question, the other two are still open.

1. *Two definitions, one object—which to formalize from.* Virtual roots carry two classical definitions: the $\mathcal{R}_d$ minimiser construction of González-Vega–Lombardi–Mahé [1] used here, and the values at which `fourierVar` jumps, the definition of Coste et al. [2]. The two agree, and *that agreement is itself Coste’s theorem*—not something this paper opens. What this paper does is formalize the construction and *bridge* it to `fourierVar` for the count (Section 6). The jumps definition would make the count almost definitional but the interlacing harder; so the genuinely open part is not the mathematics but the *Lean* choice—is re-deriving Coste’s equivalence from the jumps side cheaper, in a proof assistant, than the bridge taken here? The interlacing was free on the construction side, the count would be free on the jumps side; neither definition gives both at once.
2. *Continuity, the other half of their reason for being.* Virtual roots vary *continuously* with the coefficients—indeed González-Vega–Lombardi–Mahé prove the $\rho_{d,j}$ are semialgebraic continuous functions [1], which is half of why they introduced them. The mathematics is theirs and settled; what is open is only its formalization, untouched here. A Lean proof would have to make $\mathcal{R}_d$, a chosen minimiser, depend continuously on p ; the monotonicity bricks that pin the minimiser down are the natural starting point, but the statement is genuinely analytic and beyond the algebraic bookkeeping of this paper.
3. *One interlacing, three faces.* The Rolle interlacing places virtual roots beside the families that run through the real-rootedness literature: the roots of a polynomial and its derivative (classical Rolle [6]), Newton’s inequalities [20], and the Heilmann–Lieb matching polynomial. Eisermann’s abstract *Sturm chains* [12] already axiomatize one such interlacing structure. Is there a single Lean notion of “interlacing family” that subsumes all of these—and where does the virtual-root family, defined even when no real roots exist, sit inside it? That is the bead I would most like to carry forward.

Reproducing

```
git clone https://github.com/karlesmarin/godsil-gutman-lean
cd godsil-gutman-lean && lake exe cache get
lake build BudanFourier          # the imported sign-variation toolkit
lake env lean VirtualRoots.lean  # the structural theory
lake env lean VirtualRootsCount.lean # the exact count
```

Lean 4 (godsil toolchain pin v4.30) over Mathlib. The structural theorems are in `VirtualRoots.lean` and the exact count `card_vroots_Ioc_eq_fourierVar` in `VirtualRootsCount.lean`; their supports are Table 1. Both files are sorry-free, and `#print` axioms on `vroots_interlacing` and on `card_vroots_Ioc_eq_fourierVar` reports only `propext`, `Classical.choice`, `Quot.sound`. The worked signs of every figure ($x^2 + 1$, $x^3 - x$, $x^2 - t$) are re-derived in exact arithmetic before the figure is drawn.

References

- [1] L. González-Vega, H. Lombardi, L. Mahé, Virtual roots of real polynomials, *J. Pure Appl. Algebra* **124** (1998), no. 1–3, 147–166.
- [2] M. Coste, T. Lajous-Loaeza, H. Lombardi, M.-F. Roy, Generalized Budan–Fourier theorem and virtual roots, *J. Complexity* **21** (2005), no. 4, 479–486.
- [3] R. Descartes, *La Géométrie* (1637); the rule of signs bounds the number of positive roots by the number of sign changes of the coefficients.
- [4] F. D. Budan de Boislaurent, *Nouvelle méthode pour la résolution des équations numériques*, Paris (1807; 2nd ed. 1822).
- [5] J. Fourier, *Analyse des équations déterminées*, Firmin Didot, Paris (posthumous, 1831).
- [6] M. Rolle, *Traité d’algèbre* (1690); between two consecutive real roots of a differentiable function lies a root of its derivative.
- [7] C. Sturm, Mémoire sur la résolution des équations numériques, *Mémoires présentés par divers savants à l’Académie Royale des Sciences* **6** (1835) 271–318.
- [8] A. G. Akritas, Reflections on a pair of theorems by Budan and Fourier, *Mathematics Magazine* **55** (1982) 292–298.
- [9] G. E. Collins, A. G. Akritas, Polynomial real root isolation using Descartes’ rule of signs, in *Proc. SYMSAC ’76*, ACM, 1976, 272–275.
- [10] A. Alesina, M. Galuzzi, A new proof of Vincent’s theorem, *L’Enseignement Mathématique* **44** (1998) 219–256.
- [11] S. Basu, R. Pollack, M.-F. Roy, *Algorithms in Real Algebraic Geometry*, 2nd ed., Algorithms and Computation in Mathematics **10**, Springer, 2006 (sign-variation theory, Ch. 2, and virtual roots).
- [12] M. Eisermann, The fundamental theorem of algebra made effective: an elementary real-algebraic proof via Sturm chains, *Amer. Math. Monthly* **119** (2012), no. 9, 715–752 (abstract “Sturm chains” and the Cauchy index).
- [13] W. Li, L. C. Paulson, Counting polynomial roots in Isabelle/HOL: a formal proof of the Budan–Fourier theorem, in *Proc. CPP 2019*, ACM, 2019, 52–64; Archive of Formal Proofs entry “The Budan–Fourier Theorem” (W. Li, 2018), https://isa-afp.org/entries/Budan_Fourier.html (formalizes the inequality with multiplicity, not the virtual-root object).
- [14] C. Cohen, Construction of real algebraic numbers in Coq, in *Interactive Theorem Proving (ITP 2012)*, LNCS **7406**, Springer, 2012, 67–82 (basis of `mathcomp-real-closed`: actual roots and the Thom encoding, not virtual roots).
- [15] M. Eberl, Sturm’s Theorem, *Archive of Formal Proofs* (2014), https://isa-afp.org/entries/Sturm_Sequences.html.
- [16] The mathlib Community, The Lean mathematical library, in *Certified Programs and Proofs (CPP 2020)*, ACM, 2020, 367–381.
- [17] L. de Moura, S. Ullrich, The Lean 4 theorem prover and programming language, in *Automated Deduction (CADE 28)*, LNCS **12699**, Springer, 2021, 625–635.
- [18] C. Marín, *Counting with the Derivative Tower: the Budan–Fourier Theorem, Machine-Checked in Lean 4* (2026, companion formalization; `BudanFourier.lean`).

- [19] C. Marín, *The Staircase of Signs: Sturm's Root-Counting Theorem, Machine-Checked in Lean 4*, Zenodo, 2026, doi:10.5281/zenodo.20707348.
- [20] C. Marín, *The Inward Bow of a Real-Rooted Polynomial: Newton's Inequalities, Machine-Checked in Lean 4*, Zenodo, 2026, doi:10.5281/zenodo.20693064.